MONASH University
Information Technology

FIT5201 - Machine Learning

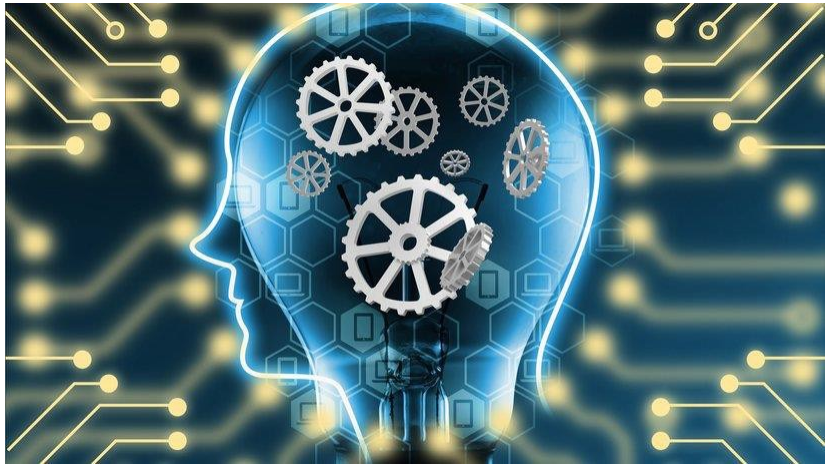
Module 1: Elements of Machine Learning

Objectives

- Describe machine learning key concepts and terminology
- Part A (Week 1):
  - Machine learning process
  - Predictive models, algorithm, parameters, data
  - How to select a model
- Part B (Week 2):
  - Probabilistic models
  - Predictive uncertainty

Part A

- Introduction to Machine Learning
- Model Selection



What is Machine Learning (ML)?

Human

Learn from Experience

Machine

Learn from Experience?

experience is **data** (sensor data, human-curated input data, etc)

Why let machines learn?

- automation of cognitive tasks
- insights into real-world processes

What is Machine Learning (ML)?

Note: We focus on statistical machine learning

ML process

1. Define variables of interest that abstract real-world domain
2. Obtain data of sampled variable values
3. Define model (or "form") of relationship between variables to learn
4. Run learning algorithm to fit model / model parameters
5. Perform model evaluation

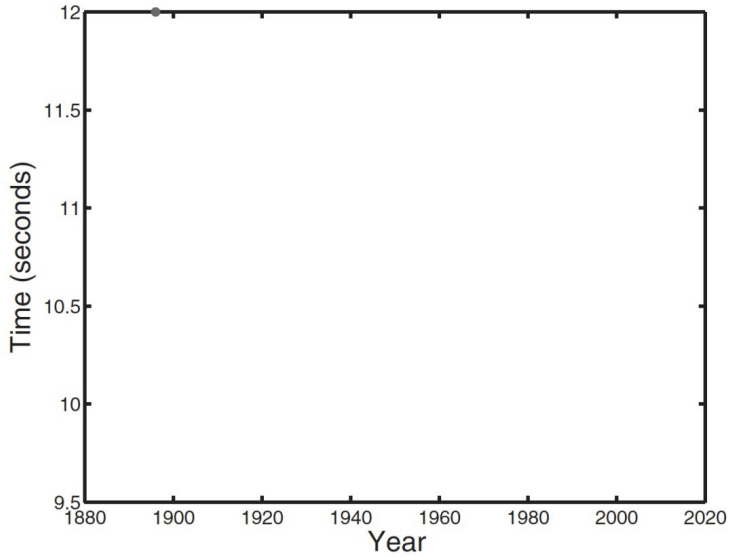
this unit

6. If good enough: apply/use depending on *purpose*

Automation: predict unknown variable values to make decisions

Insight: infer how variables affect each other

Example: Prediction of Olympic 100m gold medallist time

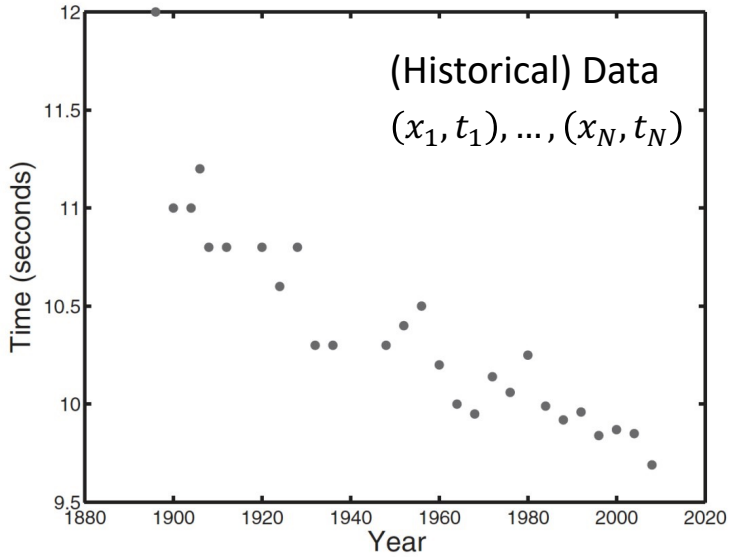


Variables of interest

Example: Prediction of Olympic 100m gold medallist time

What could be a model?

Target T



Input X

Variables of interest

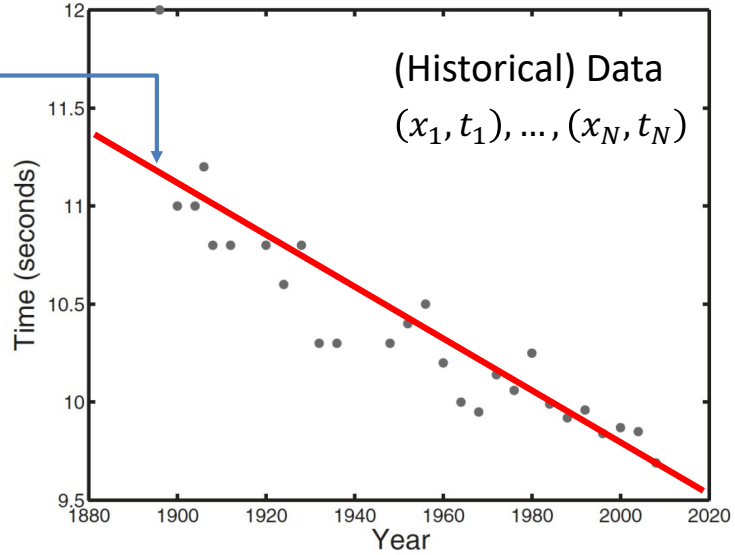
Example: Prediction of Olympic 100m gold medallist time

$T = w_1 X + w_0$

Parameters

Model

Target T

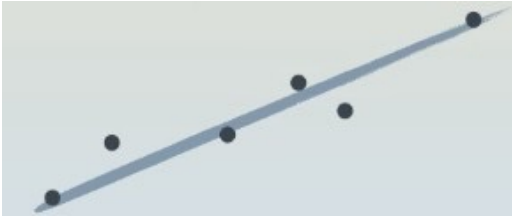


Input X

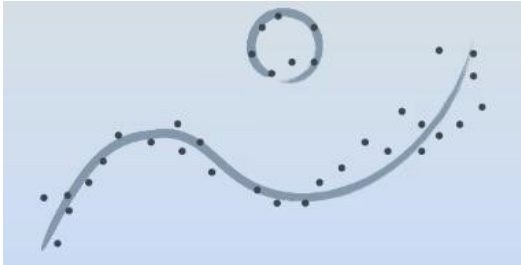
Variables of interest

Model Definition

- Types of relationships between x and t
  - Linear relationship:
    - $\mathbf{t} = \mathbf{w_0} + \mathbf{w_1 x}$ ($\mathbf{w_0}$, $\mathbf{w_1}$: model parameters)



  - Non-linear relationship:
    - $\mathbf{t} = \mathbf{w_0} + \mathbf{w_1 x} + \mathbf{w_2 x^2} + \cdots + \mathbf{w}_M \mathbf{x}^M = \sum_{\mathbf{j=1}}^{\mathbf{M}} \mathbf{w_j x^j}$



  - Model parameters are something that need to be determined somehow.

Model / Parameter Fitting

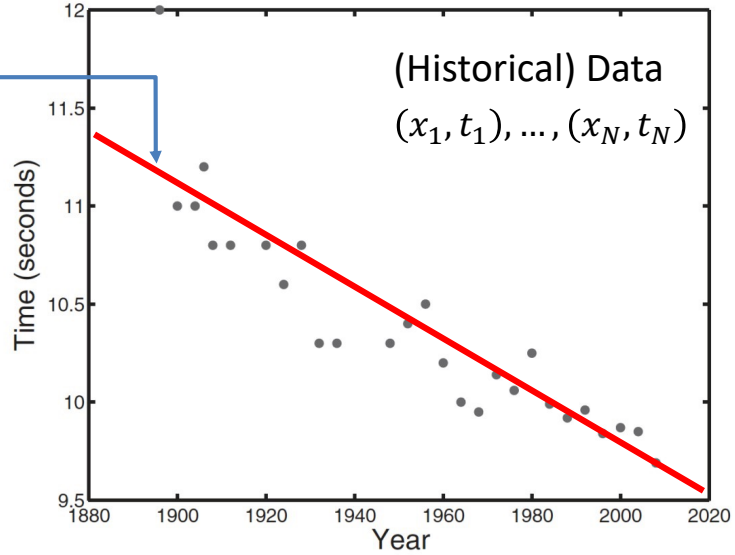
$$\boxed{T \approx y(X, \boldsymbol{w})} = w_1 X + w_0$$

Model

Parameters

quantify model performance by error (or loss) function

Target T



Input X

Variables of interest

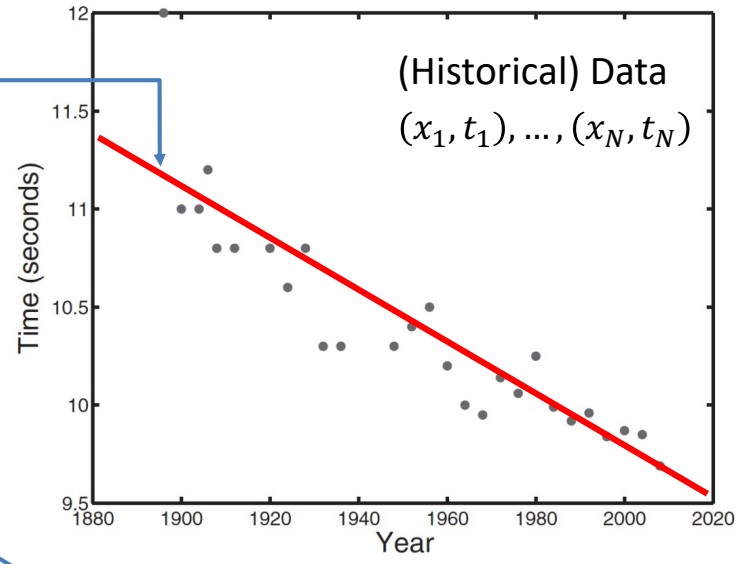
Model / Parameter Fitting

$$T \approx y(X, \boldsymbol{w}) = w_1 X + w_0$$

Model

find parameters with small generalisation error

$$\mathbb{E}\left[\frac{1}{2}\left(T - y(X, \boldsymbol{w})\right)^2\right]$$



expected value over possible (random) data

$$\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} \frac{1}{2}\left(t - y(x, \boldsymbol{w})\right)^2 p(t, x)\,dx\,dt$$

Model / Parameter Fitting

$$T \approx y(X, \boldsymbol{w}) = w_1 X + w_0$$

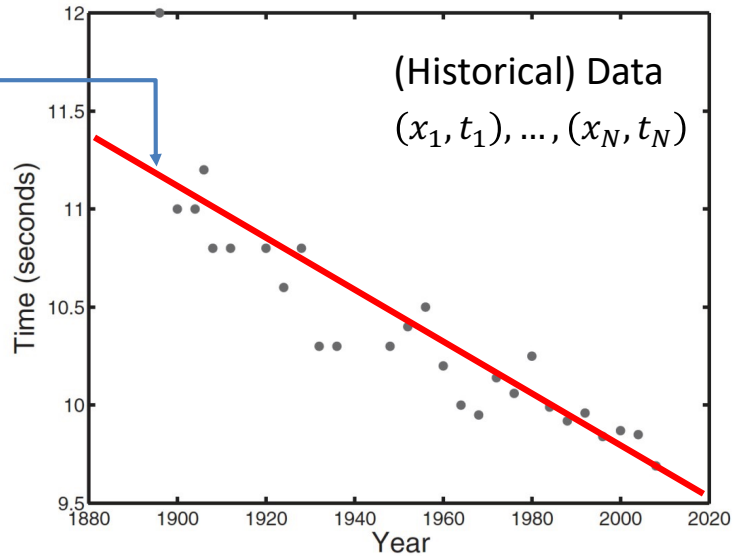
Model

find parameters with small generalisation error

$$\mathbb{E}\left[\frac{1}{2}\left(T - y(X, \boldsymbol{w})\right)^2\right]$$

by minimising training error

$$E(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\left(t_n - y(x_i, \boldsymbol{w})\right)^2$$



Model Fitting vs Performance Estimation

Goal: minimal generalisation error (prediction risk), i.e., expected value of loss function l (e.g., squared error $l(t, y) = (t - y)^2/2$)

$$\mathbb{E}\left(l\big(T, y(X, \boldsymbol{w})\big)\right)$$

Parameter learning: minimise training error (empirical risk), i.e., mean loss value on training data

$$E(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} l(t_n, y(\boldsymbol{x}_n, \boldsymbol{w}))$$

$$\boldsymbol{w}^* = \operatorname{argmin}\{E(\boldsymbol{w}) : \boldsymbol{w} \in \mathbb{R}^k\}$$

Performance estimation: want to estimate generalisation error of $\boldsymbol{w}^*$

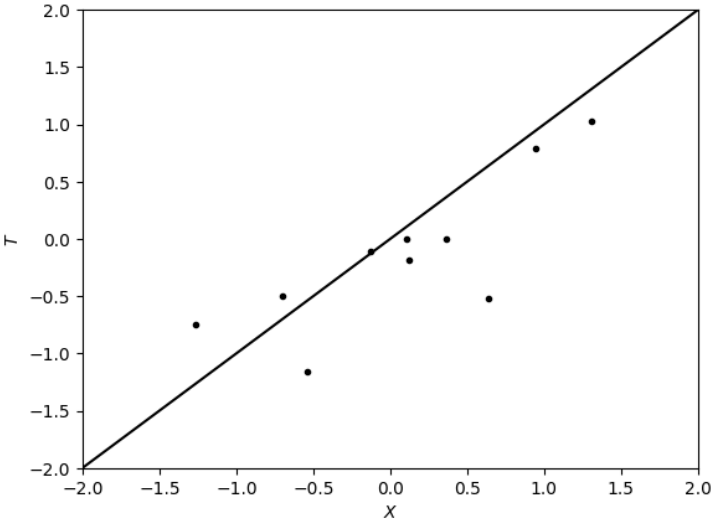
Problem: training error tends to underestimate generalisation error

$$\mathbb{E}\left[l\big(T, y(X, \boldsymbol{w}^*)\big)\right] > \mathbb{E}[E(\boldsymbol{w}^*)]$$

Optimistic Bias of Training Error

Setting: $X \sim N(0,1)$
$T = X + \epsilon$
$\epsilon \sim N(0, 0.25)$

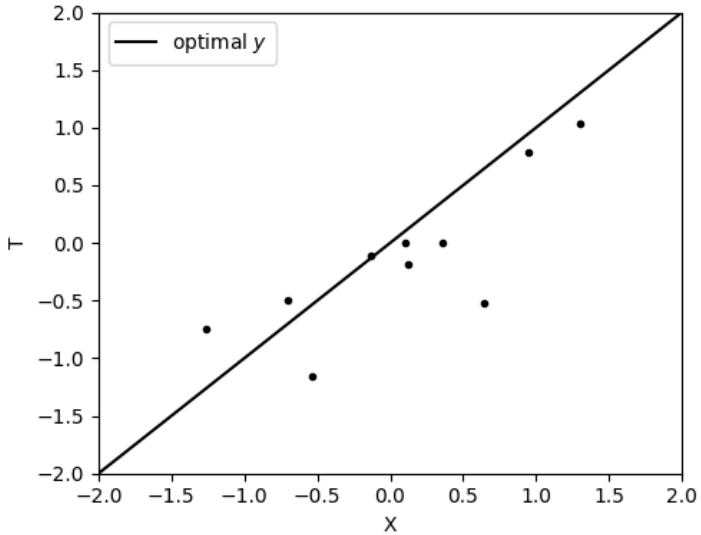
Model: $y(X, w) = wX$
single 'slope' parameter w

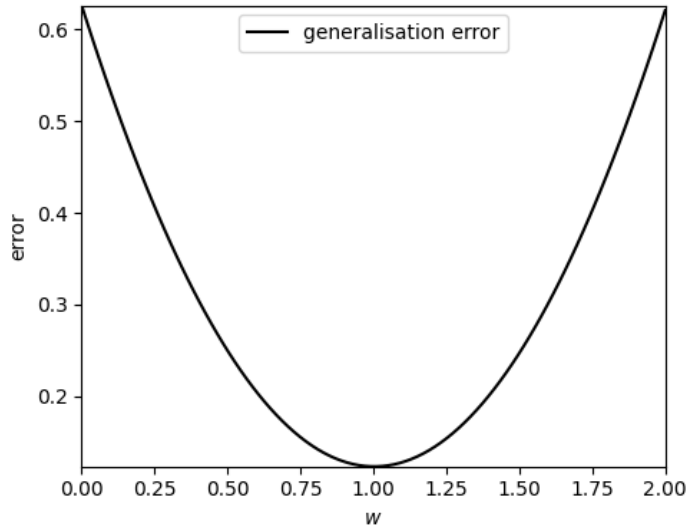


Optimistic Bias of Training Error

Setting: $X \sim N(0,1)$
$T = X + \epsilon$
$\epsilon \sim N(0, 0.25)$

Model: $y(X, w) = wX$
single 'slope' parameter w

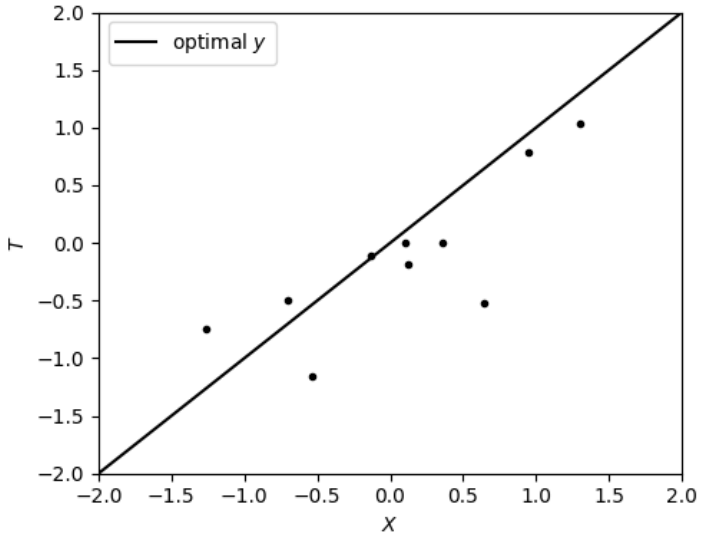


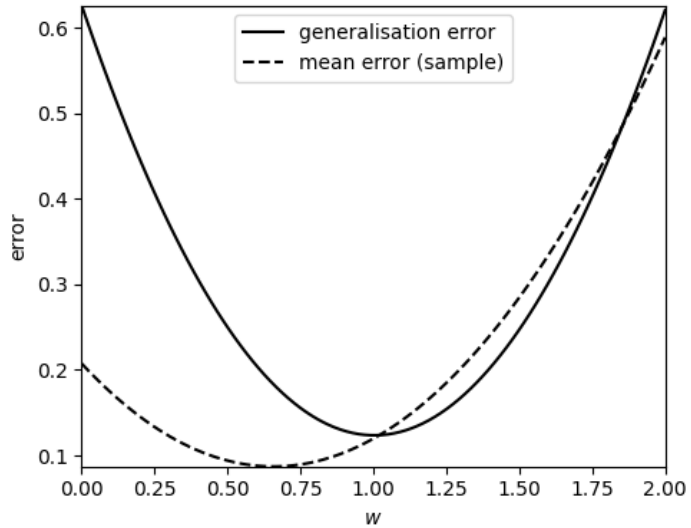


Optimistic Bias of Training Error

Setting: $X \sim N(0,1)$
$T = X + \epsilon$
$\epsilon \sim N(0, 0.25)$

Model: $y(X, w) = wX$
single 'slope' parameter w

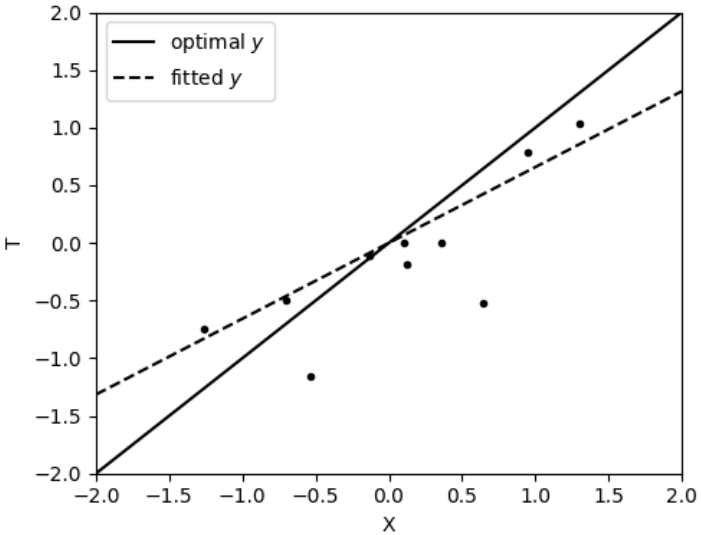


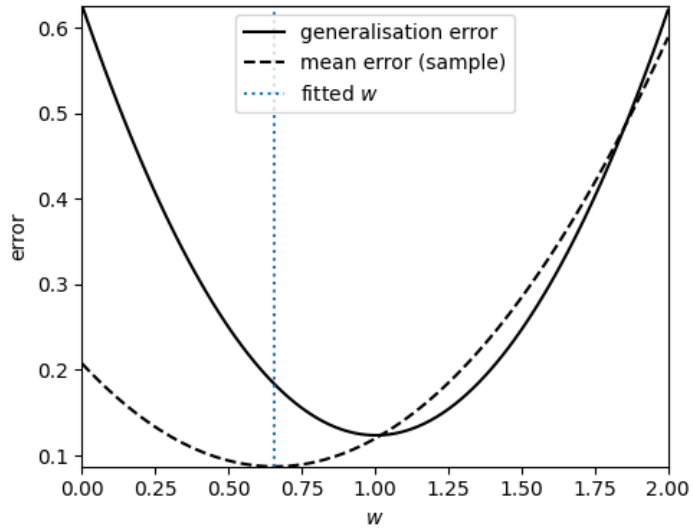


Optimistic Bias of Training Error

Setting: $X \sim N(0,1)$
$T = X + \epsilon$
$\epsilon \sim N(0, 0.25)$

Model: $y(X, w) = wX$
single 'slope' parameter w



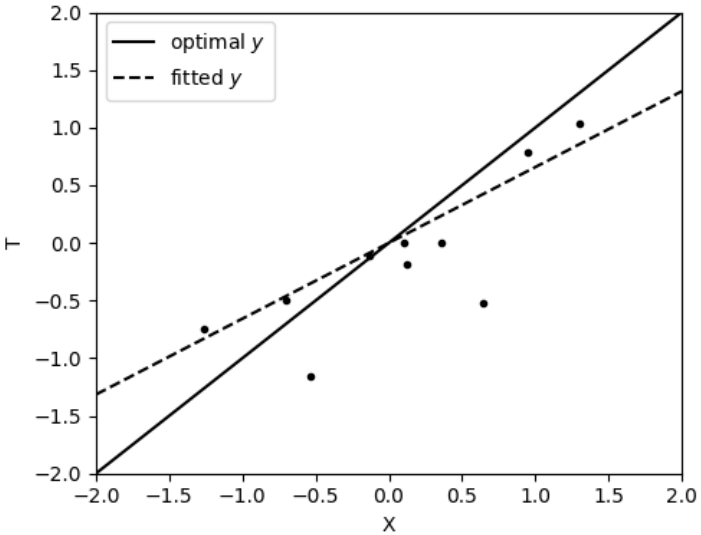


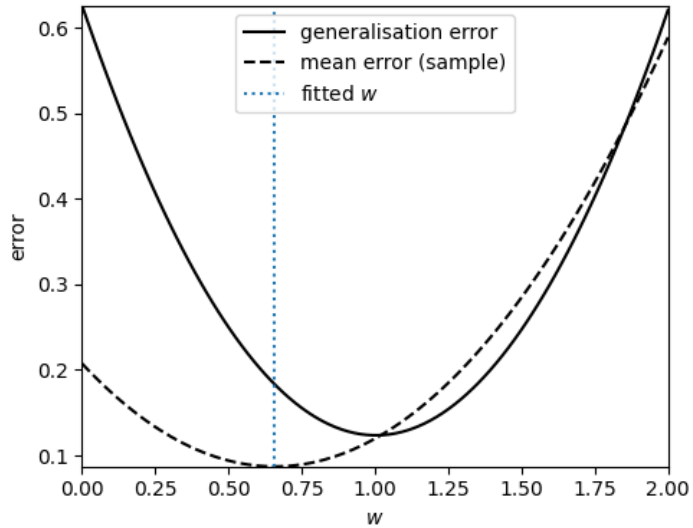
Note: Error on data *overestimates* generalisation error for some w and *underestimates* it for other.

Optimistic Bias of Training Error

Setting: $X \sim N(0,1)$
$T = X + \epsilon$
$\epsilon \sim N(0, 0.25)$

Model: $y(X, w) = wX$
single 'slope' parameter w



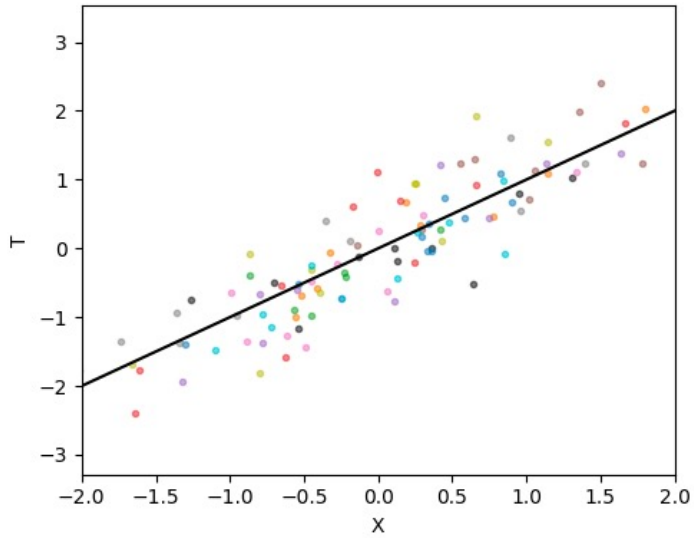


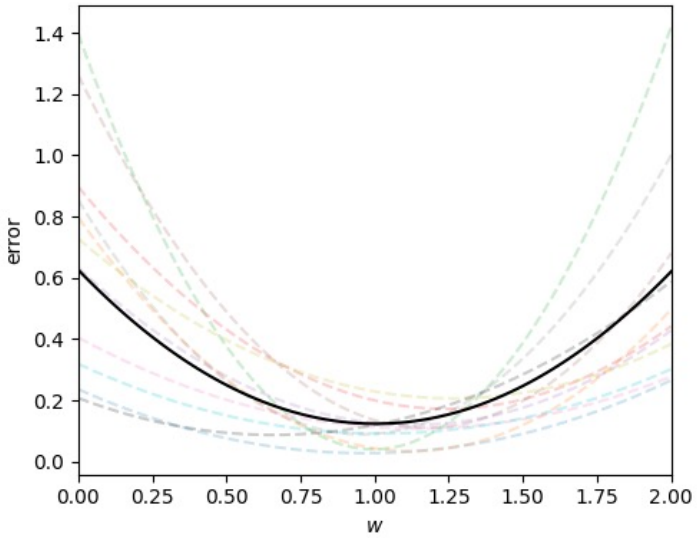
However: where error on data is minimal, it tends to *underestimate*…
…and if data is our training data -> min error on data is exactly our training error

Unbiasedness of Error on Random Data Unrelated to given Weight

Setting: $X \sim N(0,1)$
$T = X + \epsilon$
$\epsilon \sim N(0, 0.25)$

Model: $y(X, w) = wX$
single 'slope' parameter w





In contrast: If we first fix any w and then sample a random dataset -> <u>no bias</u>

Unbiased Performance Estimate via Train/Test Split

$$E(w) = \frac{1}{N}\sum_{n=1}^{N} l(t_n, y(\boldsymbol{x}_n, \boldsymbol{w}))$$

$$\boldsymbol{w}_N^* = \operatorname{argmin} E(w)$$

$$\mathbb{E}[E(\boldsymbol{w}_N^*)] < \mathbb{E}\left[l\left(T, y(X, \boldsymbol{w}_N^*)\right)\right]$$

Problem: train error *tends* to underestimate generalisation error,
(optimistically biased estimate)

Unbiased Performance Estimate via Train/Test Split

$$E(w) = \frac{1}{N}\sum_{n=1}^{N} l(t_n, y(\boldsymbol{x}_n, \boldsymbol{w}))$$

$$\boldsymbol{w}_N^* = \operatorname{argmin} E(w)$$

$$\mathbb{E}[E(\boldsymbol{w}_N^*)] < \mathbb{E}\big[l\big(T, y(X, \boldsymbol{w}_N^*)\big)\big]$$

$(\boldsymbol{x}_1, t_1), \ldots$ $\ldots, (\boldsymbol{x}_N, t_N)$

shuffle and split

train set

test set

$(\boldsymbol{x}_1', t_1'), \ldots$ $\ldots, (\boldsymbol{x}_M', t_M')$ $\ldots, (\boldsymbol{x}_N', t_N')$

$$E_{\text{train}}(\boldsymbol{w}) = \frac{1}{M}\sum_{n=1}^{M} l\big(t_n', y(\boldsymbol{x}_n', \boldsymbol{w})\big)$$

$$E_{\text{test}}(\boldsymbol{w}) = \frac{1}{N-M}\sum_{n=M+1}^{N} l\big(t_n', y(\boldsymbol{x}_n', \boldsymbol{w})\big)$$

$$\boldsymbol{w}_M^* = \operatorname{argmin} E_{\text{train}}(\boldsymbol{w})$$

$$\mathbb{E}[E_{\text{test}}(\boldsymbol{w}_M^*)] = \mathbb{E}\big[l\big(T, y(X, \boldsymbol{w}_M^*)\big)\big]$$

test error unbiased estimate of gen. error of parameters trained on train set

Unbiased Performance Estimate via Train/Test Split

$$E(w) = \frac{1}{N}\sum_{n=1}^{N} l(t_n, y(\boldsymbol{x}_n, \boldsymbol{w}))$$

$$\boldsymbol{w}_N^* = \operatorname{argmin} E(w)$$

$$\mathbb{E}[E(\boldsymbol{w}_N^*)] < \mathbb{E}\big[l\big(T, y(X, \boldsymbol{w}_N^*)\big)\big]$$

$(\boldsymbol{x}_1, t_1), \ldots$ $\ldots, (\boldsymbol{x}_N, t_N)$

shuffle and split

train set

test set

$(\boldsymbol{x}_1', t_1'), \ldots$ $\ldots, (\boldsymbol{x}_M', t_M')$ $\ldots, (\boldsymbol{x}_N', t_N')$

$$E_{\text{train}}(\boldsymbol{w}) = \frac{1}{M}\sum_{n=1}^{M} l\big(t_n', y(\boldsymbol{x}_n', \boldsymbol{w})\big)$$

$$E_{\text{test}}(\boldsymbol{w}) = \frac{1}{N-M}\sum_{n=M+1}^{N} l\big(t_n', y(\boldsymbol{x}_n', \boldsymbol{w})\big)$$

$$\boldsymbol{w}_M^* = \operatorname{argmin} E_{\text{train}}(\boldsymbol{w})$$

$$\mathbb{E}[E_{\text{test}}(\boldsymbol{w}_M^*)] = \mathbb{E}\big[l\big(T, y(X, \boldsymbol{w}_M^*)\big)\big] > \mathbb{E}\big[l\big(T, y(X, \boldsymbol{w}_N^*)\big)\big]$$

test error conservative estimate of gen. error of parameters trained on all data

Considerations for Train/Test Split

Notes:

- Always split intro training and test
- May or may not ”refit” model using all data
- How to choose split proportions?

Large train, small test

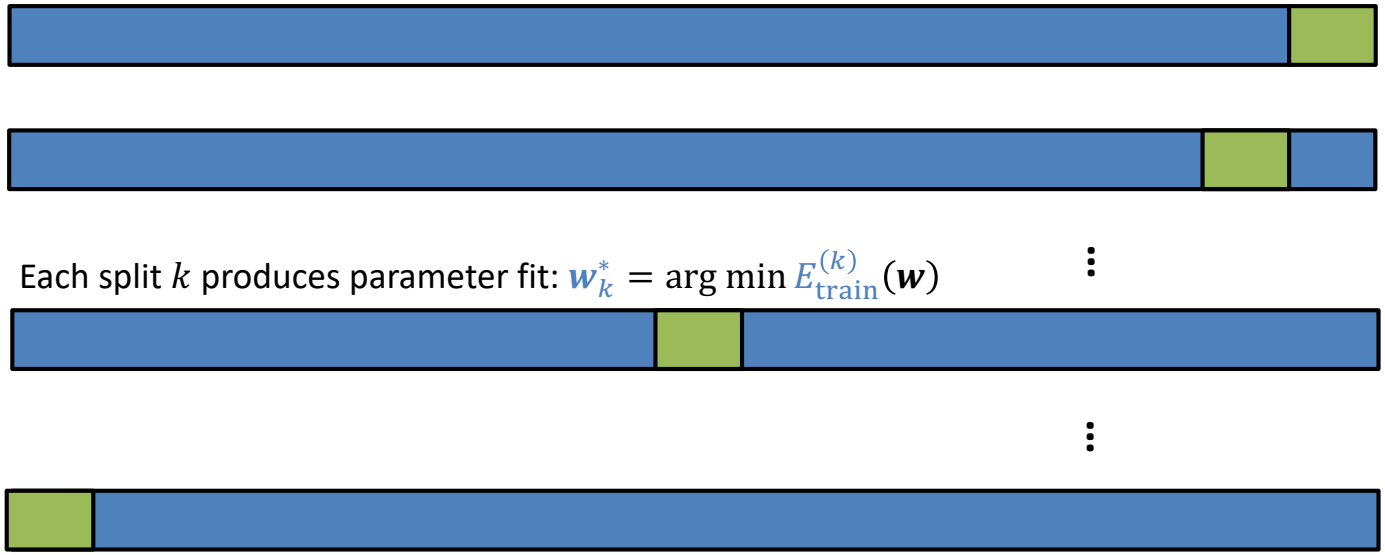
- Better “initial” fit $\boldsymbol{w}_M^*$, smaller difference $\mathbb{E}\left[l\left(T, y(X, \boldsymbol{w}_M^*)\right)\right] - \mathbb{E}\left[l\left(T, y(X, \boldsymbol{w}_N^*)\right)\right]$
- However, unreliable performance estimate due to high variance $\mathbb{V}[E_{\text{test}}(w_N^*)]$

Small train, large test

- Reliable low variance performance estimate
- However likely poor fit $\boldsymbol{w}_M^*$ and large difference $\mathbb{E}\left[l\left(T, y(X, \boldsymbol{w}_M^*)\right)\right] > \mathbb{E}\left[l\left(T, y(X, \boldsymbol{w}_N^*)\right)\right]$

Cross Validation for Unbiased and Low Variance Performance Estimate

Idea: use K different non-overlapping test sets and each complement to train



Each split k produces parameter fit: $\boldsymbol{w}_k^* = \arg\min E_{\text{train}}^{(k)}(\boldsymbol{w})$

Cross validation error is average of unbiased test errors: $E_{\text{CV}} = \sum_{k=1}^{K} E_{\text{test}}^{(k)}(\boldsymbol{w}_k^*)/K$

Summary

Elements of statistical learning

- (Random) variables that abstract real-world process/system
- Data sampled from that process
- Model with parameters that relate variables
- Learning algorithm to find parameter values to minimise training error (measured with training data)
- Estimation of generalisation error with test data

Key questions

- How to select model
- How to efficiently learn parameters

ML tasks differ based on variables

Supervised learning

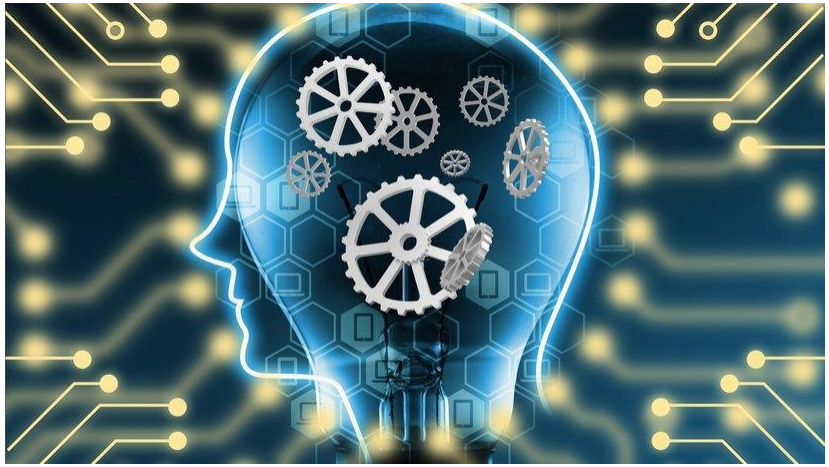
- Variables partitioned into input variables X and target variables T
- Values for both are given in training data
- Regression: target variable(s) real-valued and continuous
- Classification: target variable(s) take on finite set of values

Unsupervised learning

- Only one kind of variable in training data
- Clustering: group future/present observations (can be regarded as problem with unobserved target variable)
- Outlier detection / density estimation: no target variable, just model all variables jointly

Part A

- Introduction to Machine Learning
- **Model Selection**

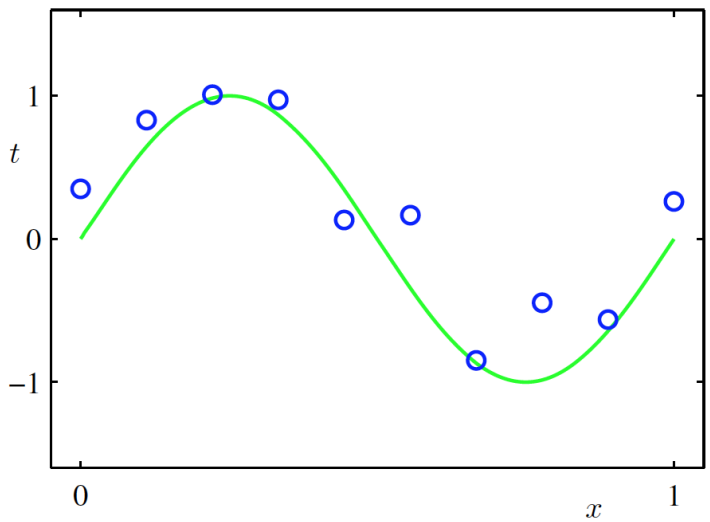


Another regression problem example

Data sample $(x_1, t_1), \ldots, (x_N, t_N)$ with $N = 10$

…each independent realisations of X and T that have distribution:

$$T = \sin 2\pi X + \varepsilon$$
$$\varepsilon \sim \mathrm{N}(0, \sigma^2)$$
$$X \sim \mathrm{U}(0,1)$$



- circle: training points
- curve: the sine function used to generate the training data

Recap objectives

- To use the training set to build a model that can predict the value of t for a new input x accurately, without knowledge of the green curve.

- This involves implicitly trying to discover the underlying function

$$t = sin(2\pi x)$$

- To achieve good "generalisation" of the model by making accurate predictions for new data.

- Assess the generalisation of the trained model by comparing the predicted value and original value of for each input in test set.

Learning a Model

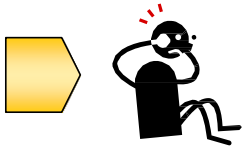
- Consider a simple approach based on curve fitting, a degree M-polynomial function:
  - $y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=1}^{M} w_j x^j$, where
    - $\mathbf{w}$ is the vector denoting the model parameters collectively (i.e. polynomial coefficients);
    - M is the order of the polynomial;
    - x^j denotes x raised to the power of j.
  - Note that $y(x, \mathbf{w})$ is a non-linear function of x, but a linear function of the coefficients $\mathbf{w}$. We call the function $y(x, \mathbf{w})$ a linear model
- Question: how to determine $\mathbf{w}$?

Learning a Model

- How to determine $\mathbf{w}$?
  - When fitting the polynomial to the training set, we need to find $\mathbf{w}$ that minimise an error function that measures the misfit between $y(x, \mathbf{w})$, for any given value of $\mathbf{w}$, and the training set data points.

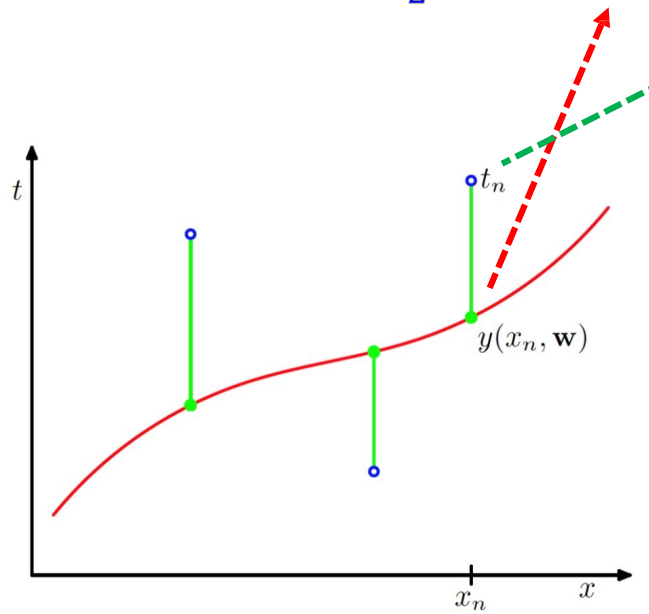
- How to define such an error function?

$$E(\mathbf{w}) := \frac{1}{2}\sum_{n=1}^{N} [\underbrace{y(x_{n,}\mathbf{w})}_{predictions} - \underbrace{t_n}_{\substack{target \\ values}}]^2$$



Learning a Model

- Error Function: $E(\mathbf{w}) := \frac{1}{2}\sum_{n=1}^{N}[y(x_n, \mathbf{w}) - t_n)]^2$



$E(\mathbf{w})$ means "the sum of the squares of the errors" between the predictions $y(x, \mathbf{w})$ for each data point x_n and the target values t_n.

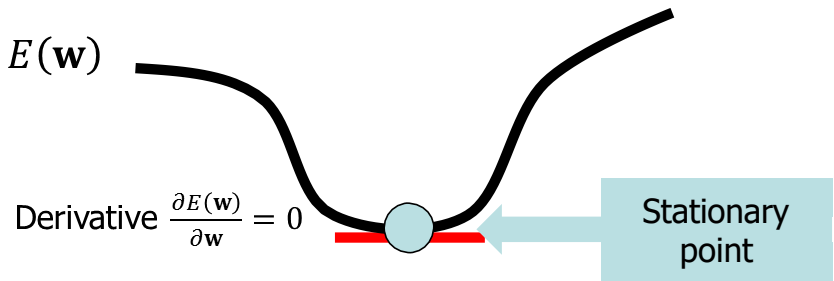
Learning a Model

- Training Objective: find $\mathbf{w}\ (w_0, \dots w_M)$ that minimise the error function

$$E(\mathbf{w}) := \frac{1}{2}\sum_{n=1}^{N}[y(x_{n,}\mathbf{w}) - t_n)]^2$$

$\mathbf{0}$? Perfectly accurate model

- Optimisation Algorithm: Learning problem is solved by choosing the value of $E(\mathbf{w}^*)$: $\mathbf{w}^* := \arg\min_{\mathbf{w}} E(\mathbf{w})$



Learning a Model

- Linear models
  - The error function for linear models is quadratic of **w**
  - Its derivatives with respect to **w** is linear
  - The minimization of the error function has a unique solution
  - Learning process much easier

Model complexity

- Consider again the polynomial function:

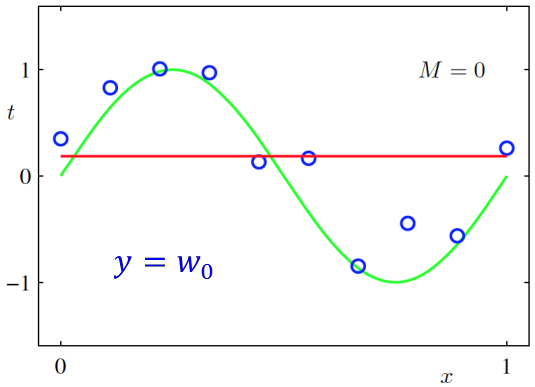
$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_{\boldsymbol{M}}\, x^{\boldsymbol{M}} = \sum_{j=1}^{\boldsymbol{M}} w_j x^j$$

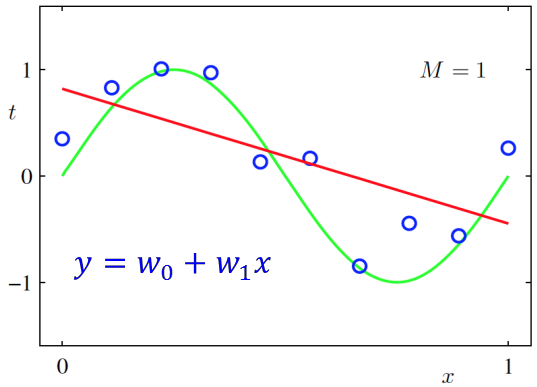
- Determines the complexity of the model

M

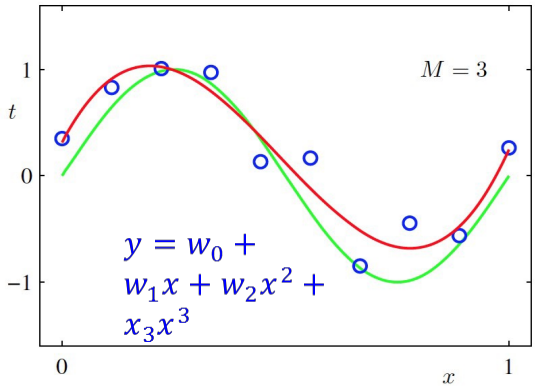
The higher the order, the more complex the model.

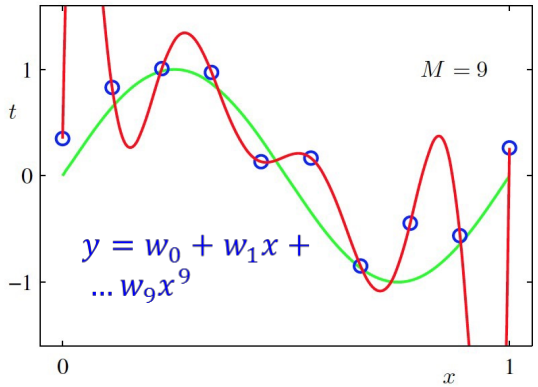
Fitting polynomial with M to the data





M=0, 1: poor fits to the data, thus poor representation of $\sin(2\pi x)$

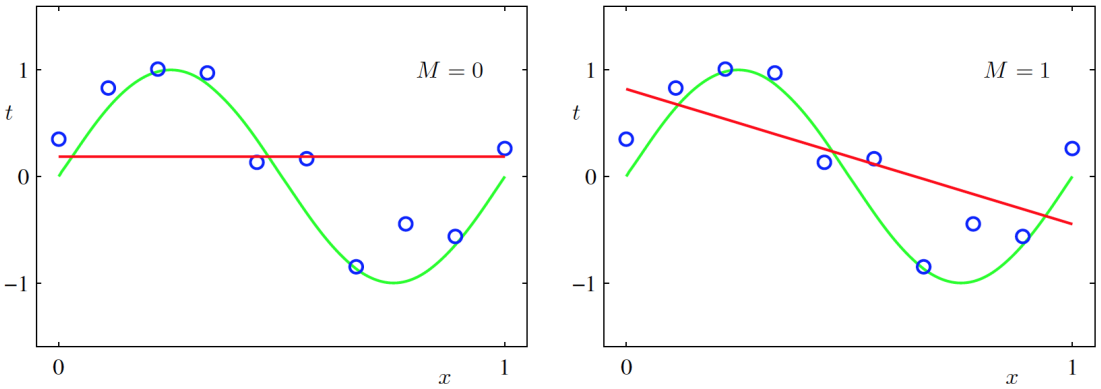




M=3: well fit to $\sin(2\pi x)$

M=9: an excellent fit to the training

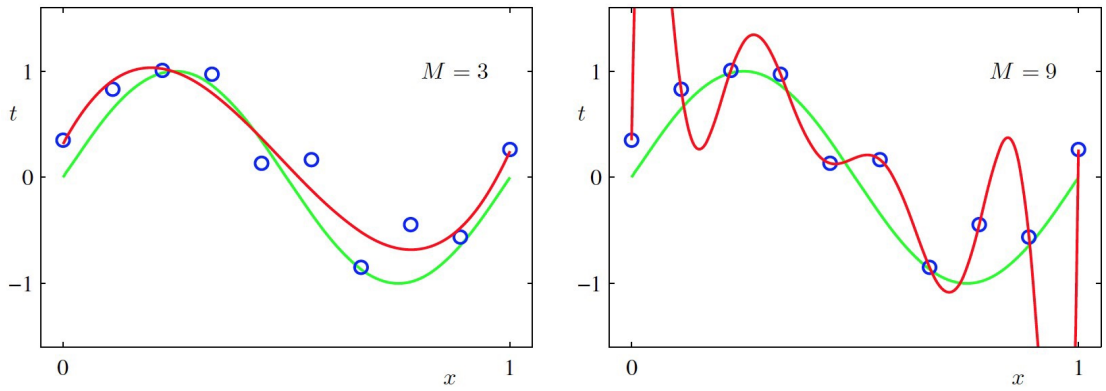
Too simple: Under-fitting



- Both model with M=0 and M=1 have **poor representation of $\sin(2\pi x)$: "Under-fitting"**

Too complex: Over-fitting

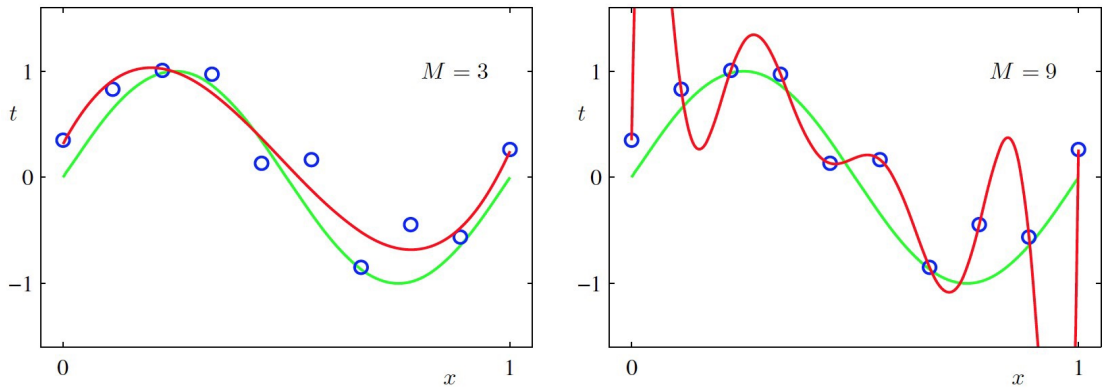
Which model is better?



- The model with M=9 has an excellent fit to the training data but **poor representation of $\sin(2\pi x)$: "Over-fitting"**

Too complex: Over-fitting

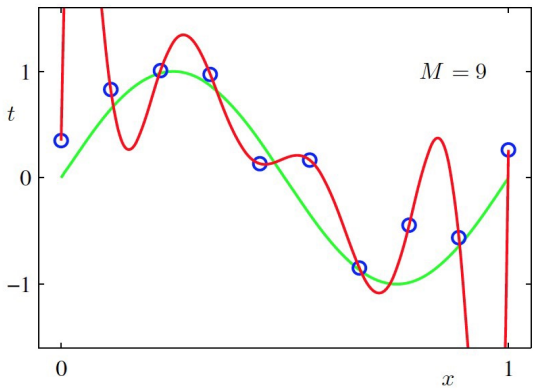
Which model is better?



- The model with M=9 has an excellent fit to the training data but **poor representation of si**
  "Over-fitting"

Cannot tell from the training error without the real function! Then how?

Generalisation Performance

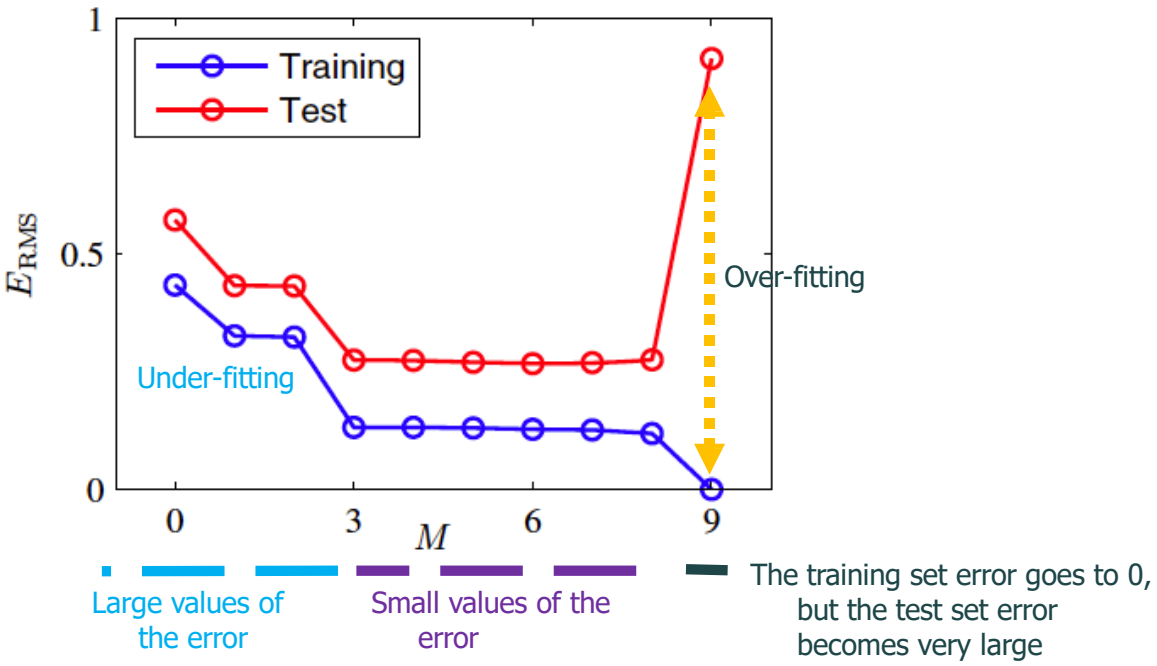


Problem:
generalisation error much larger than estimated by training error

$$\mathbb{E}\left[\frac{1}{2}\left(T - y(X, \boldsymbol{w})\right)^2\right] >> \sum_{n=1}^{N} \frac{1}{2N}\left(t_n - y(x_n, \boldsymbol{w})\right)^2$$

Solution:
Unbiased estimate of generalisation error by using test data $(x_1', t_1'), \ldots, (x_{N'}', t_{N'}')$ not involved in training

Generalisation Performance



Paradox on the model with M=9

A power series expansion (e.g. $\mathrm{y}(\mathrm{x}, \mathbf{w}) = \mathrm{w}_0 + \mathrm{w}_1\mathrm{x} + \mathrm{w}_2\mathrm{x}^2 + \cdots + \mathrm{w}_\mathrm{M}\,\mathrm{x}^\mathrm{M}$) of $\mathbf{sin}(\boldsymbol{2\pi x})$ contains all lower order polynomials.

Expect that the test error should decrease gradually as the degree **M** is increased. However, the model with M=9 shows a **large over-fitting problem.**

What's wrong with the model with M=9 then?

Paradox on the model with M=9

	$M=0$	$M=1$	$M=6$	$M=9$
$w_0^\star$	0.19	0.82	0.31	0.35
$w_1^\star$		-1.27	7.99	232.37
$w_2^\star$			-25.43	-5321.83
$w_3^\star$			17.37	48568.31
$w_4^\star$				-231639.30
$w_5^\star$				640042.26
$w_6^\star$				-1061800.52
$w_7^\star$				1042400.18
$w_8^\star$				-557682.99
$w_9^\star$				125201.43

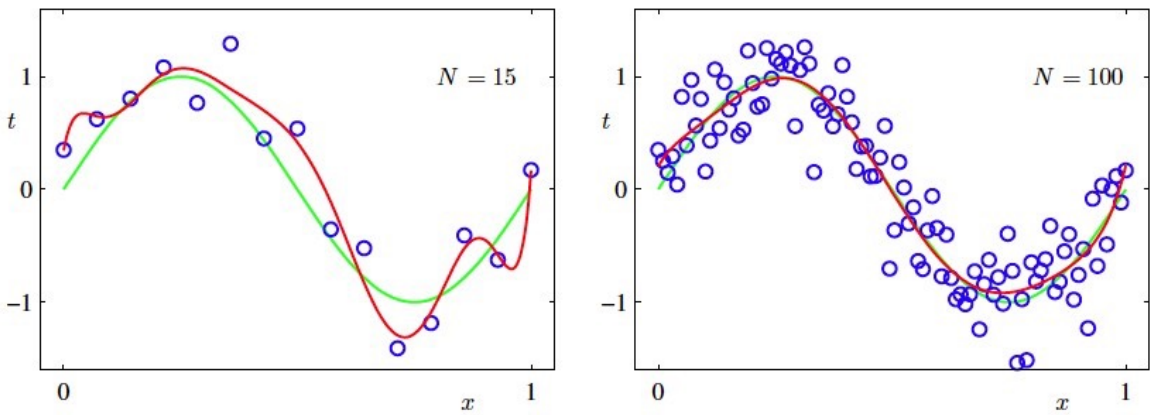
Table of the parameters for polynomials for various orders.

As M increases, the parameters become larger → The model is too flexible; and the parameters were very finely tuned to the 10 training points.

How to reduce high flexibility of the model with M = 9?

This questions is formulated as how to reduce an **over-fitting** problem?

- **Increase the size of a training set**



Plots of the solutions obtained by minimizing the error function with M = 9 for N = 15 data points (left plot) and N = 100 data points (right plot).

Regularisation

- A technique to control the over-fitting phenomenon
  - **Idea:** Add a penalty term to the error function to discourage the parameters from reaching large values:

$$E(\mathbf{w}) := \frac{1}{2}\sum_{n=1}^{N}[y(x_{n,}\mathbf{w}) - t_n)]^2 + penalty(\mathbf{w})$$

  - **Penalty Term**: the sum of square of all parameters:

$$penalty(\mathbf{w}) := \frac{\lambda}{2}\,||\mathbf{w}||^2,$$

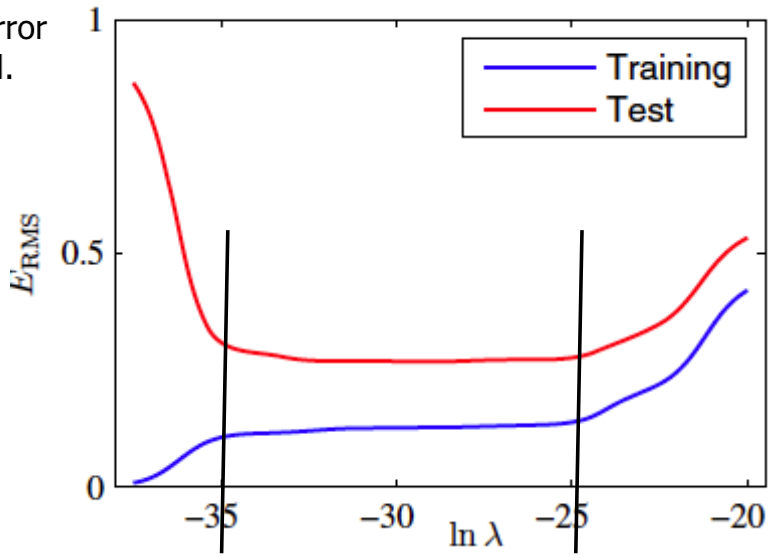
where $||\mathbf{w}||^2 = w_0^2 + w_1^2 + \ldots + w_M^2$, and $\boldsymbol{\lambda}$ is the regularization parameter governing the relative importance of the penalty term.

- <u>Revise the training objective</u>
  - tradeoff between the empirical error and the model complexity.

How does regularisation work?

For a large λ, models with high complexity can be ruled out. For a small λ, models with high training errors can be ruled out. The optimal solution lies somewhere in the middle.

Graph of the root-mean-square error vs. $\ln(\lambda)$ for the M = 9 polynomial.

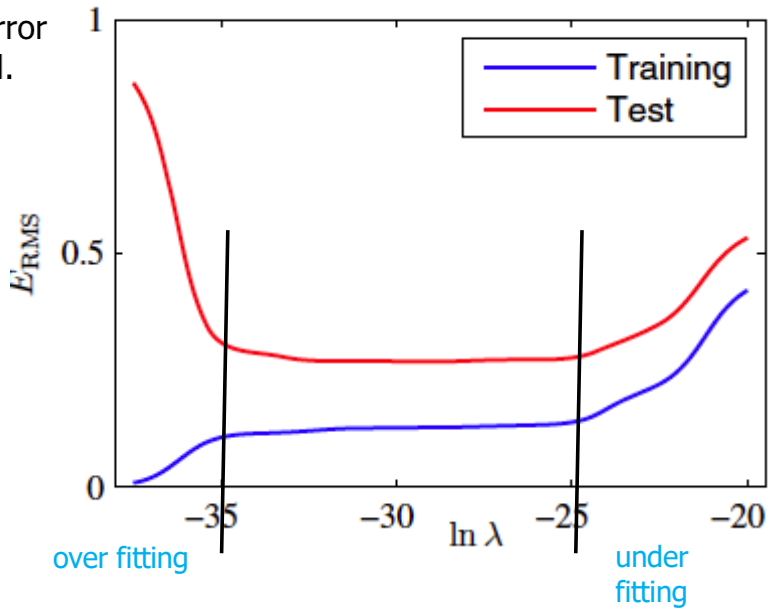


How does regularisation work?

For a large λ, models with high complexity can be ruled out. For a small λ, models with high training errors can be ruled out. The optimal solution lies somewhere in the middle.

Graph of the root-mean-square error vs. $\ln(\lambda)$ for the M = 9 polynomial.

1. λ increases, model becomes less complex, training error increases
2. λ is very small, model too complex, test error high (over fitting)
3. λ increases, model becomes less complex, testing error decreases
4. λ is too large, model too simple, testing error increases (under fitting)



Model Selection

- Revisit what we've learned from the polynomial curve fitting problem:
  - The order controls the complexity
  - The regularization parameter also controls the complexity
  - For more complex models, more parameters
  - How to train these parameters?
    - No access to the test data when we train the model, remember?
    - Cannot do this based on testing error

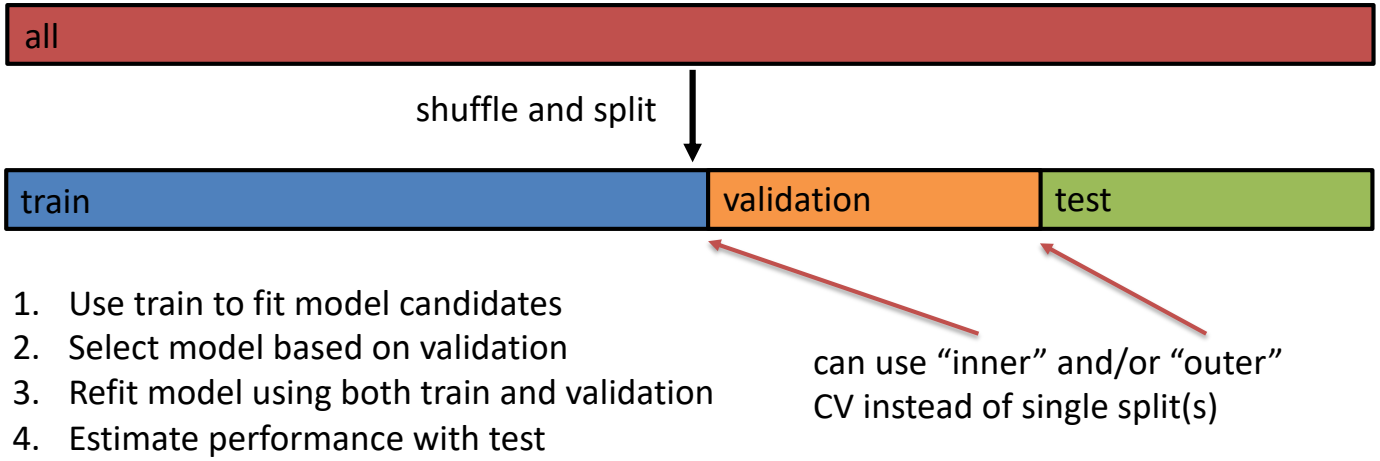
Model Selection and Performance Estimation

Scenario: We choose the degree three polynomial model after comparing its test error to that of the other degrees up to 10.

Question: Can we assume that the test error will reflect its future performance?

Answer: No, because final prediction function not independent of test data. (technically, we have "fitted" M to the test data.

Solution: Perform 3-way split.



1. Use train to fit model candidates
2. Select model based on validation
3. Refit model using both train and validation
4. Estimate performance with test

Module 1: Elements of Machine Learning

- Module Objectives
  - Provide an introduction to machine learning theory
  - **Part A (Week 1):**
    - Understand the machine learning process
    - Understand key concepts of machine learning
    - Understand how to select a good learning model

Wrap up the lecture

(Supervised) machine learning

- Learn relationship between input and target variables from data and use it to predict target values of unknown data

Challenge

- Need to determine model parameters
- Usually by minimizing an error function between predicted and correct target values, however, need to
- Choose model that does not overfit (penalty, polynomial order)

How to select model

- Hold-out test data to estimate generalisation error
- Using a cross-validation to improve estimate

Laboratory (Week 1)

- Learn how **k-Nearest Neighbors (k-NN) classifier** works
- Investigate effect of hyper-parameter k on model complexity, overfitting, and underfitting
- Using k-NN, practice some of the basic concepts of machine learning in **Python** using some standard libraries (numpy, scikit-learn)

What will we learn in Week 2

- **Part B (Week 2):**
  - Understand probabilistic machine learning
  - Understand prediction uncertainty and develop tools (**bootstrapping**) to measure it (**Tutorial**)

Thank You